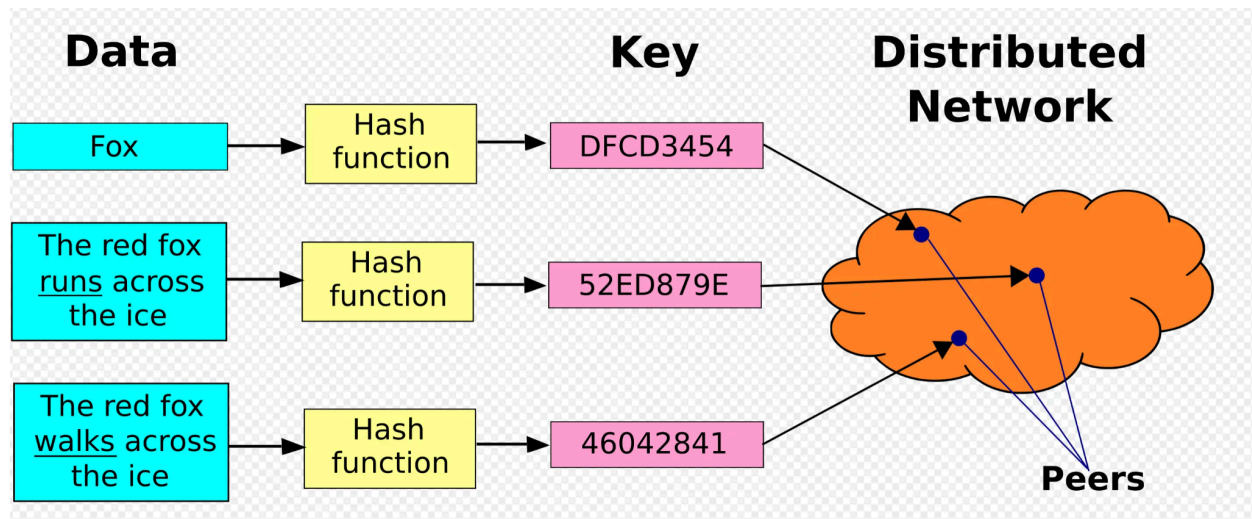




1. Project Overview

This project focuses on leveraging the provided **Python Chord DHT implementation** for a peer-to-peer (P2P) file-sharing application. The goal is to evolve the foundational DHT and file-transfer mechanism into a more robust, secure, and user-friendly system, meeting the academic requirements for a decentralized application. The final system will serve as a complete demonstration of the Chord protocol's operation, fault tolerance, and application-layer integration.

2. Objectives and Constraints

Core Objectives:

- **Establish Chord Foundation:** Ensure the existing Chord protocol logic (node join, file lookup/routing, stabilization, finger table management) is fully debugged and functional.
- **Implement Security Layer:** Integrate essential security features, specifically file integrity verification and a mechanism for end-to-end encryption.
- **Enhance Robustness:** Improve the core DHT's fault tolerance beyond basic successor pinging.
- **Modularize Code:** Separate the core networking and DHT logic from the application-level (UI/File I/O) logic for clarity and learning.

Constraints:

- **Code Base:** Must be built upon the provided `Node.py` structure.
 - **Simplicity Focus:** While advanced features are added, the implementation must be kept as clear as possible to aid in understanding the core Chord concepts.
-

3. Scope of Work (Phase I: Core Functionality & Robustness)

This phase focuses on completing and hardening the DHT layer.

3.1 Chord Protocol Improvements:

- **Predecessor Stabilization:** Implement periodic checks (e.g., `stabilize` and `notify` routines) to ensure correct pointer alignment between nodes, completing the Chord stabilization algorithm.
- **Successor List Implementation:** Add a **successor list** to each node to provide immediate failover redundancy if the primary successor disconnects abruptly.
- **Optimized Finger Table Repair:** Replace the full finger table re-computation with an incremental `fix_fingers` routine to reduce lookup overhead and distribute maintenance work.

3.2 File Management:

- **Data Migration on Join:** Implement the logic to automatically transfer relevant files (those whose keys now map to the new node) from the successor when a new node joins the network.
 - **Metadata Storage:** Instead of just a filename list, store rich file metadata (owner ID, file size, hash, etc.) at the Chord-determined node.
-

4. Scope of Work (Phase II: Security and User Experience)

This phase integrates the application-specific requirements.

4.1 Security and Integrity:

- **File Integrity Check:** Before an upload, calculate a hash (e.g., SHA-256) of the file; the recipient must verify this hash upon download to confirm transfer integrity.
- **End-to-End Encryption (E2EE) Placeholder:** Implement a **stub** for E2EE (e.g., using Python's `cryptography` library) where the file is encrypted before being sent and decrypted upon receipt. This will be an *application-level* implementation, not a replacement for the underlying TCP connection.

4.2 Application Layer:

- **Basic User Profiles:** A basic system where peers register a unique **Username/Public Key** and have a profile ID mapped to their node ID. This is a first step toward user authentication.
 - **Enhanced Menu:** Improve the command-line interface with clearer output and better handling of user inputs, especially for file management operations.
-

5. Implementation Modules

The project will be logically divided into the following modules within the existing `Node.py` or separated files:

1. **Networking_Layer:** Contains all `socket` creation, `send/recv` operations, and `pickle` serialization.
 2. **DHT_Protocol:** Contains `getHash`, `getSuccessor`, `lookupID`, `stabilize`, `notify`, `fix_fingers`, and all logic related to `pred/succ/fingerTable` updates.
 3. **File_Manager:** Contains `uploadFile`, `downloadFile`, `sendFile`, `receiveFile`, file integrity checks, and file encryption/decryption routines.
 4. **App_Interface:** Contains `printMenu`, `asAClientThread`, and user input handling.
-

6. Deliverables

- Fully functional **Secure Chord DHT File Sharing Application** written in Python.
- Demonstrable P2P file transfer with **Fault Tolerance** (nodes leaving/failing).
- Code documentation explaining the added **Stabilization** and **Security** mechanisms.
- A final report detailing the design choices and evaluation of the Chord implementation.